Lecture 3 · Part I · Lecture notes

Honest evaluation, and classification metrics

Dataset: California housing, then MNIST

How to use these notes

The primary text for this lecture is Géron, Chapters 2 and 3. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 Where we are **2**

2 The normal equation **3**

2.1 What the stationarity condition says geometrically . . . 3

2.2 When it breaks . . . 3

2.3 What Scikit-Learn actually does . . . 4

3 Measuring generalisation **4**

3.1 A number that means nothing . . . 4

3.2 "Then use the test set" . . . 5

3.3 Leakage: the same error, one step earlier . . . 5

4 Cross-validation and pipelines **6**

4.1 From holdout to k-fold . . . 6

4.2 Ten folds, not one number . . . 6

4.3 Pipelines make leakage structurally impossible 6

5 The test set, once **7**

6 A classification task, and a first detector **7**

6.1 The brief 7

6.2 The data 7

6.3 The obvious metric, and the headline number 8

6.4 Before you commit: what does nothing score? 8

7 The mathematics: what nothing scores, and what precision does **8**

7.1 The classifier that never fires 8

7.2 What the detector computes 9

7.3 Recall is monotone in the threshold; precision is not 9

8 What 96.90% was hiding **10**

8.1 The confusion matrix 10

8.2 Precision, recall and F_1 10

9 The threshold, and the curves **11**

9.1 You cannot have both 11

9.2 Choosing a threshold deliberately 11

9.3 ROC, and when not to use it 11

9.4 A better classifier 12

10 The notebook **12**

11 Design exercise: a payments app's real-time fraud engine **13**

12 Exercises **15**

Summary **15**

---

1 Where we are

Lecture 2 sealed a stratified test set of 4,128 California districts, explored the 16,512 training districts, and set up least squares: minimise $\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$, whose gradient vanishes where $\mathbf{X}^\intercal\mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{X}^\intercal y$, and whose convexity makes that stationary point the global minimum. Nothing

had been fitted and nothing scored.

This lecture has two parts. The first solves that equation, then makes the housing model's error honest: why a training score means nothing, what leakage is, holdout and k-fold cross-validation, pipelines, and the single use of the test set. The second turns to a classification task, where the obvious metric is correctly measured and nearly worthless.

2 The normal equation

From the stationarity condition to a closed form

Setting the gradient of the MSE to zero gives $\mathbf{X}^\intercal\mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{X}^\intercal y$. When $\mathbf{X}^\intercal\mathbf{X}$ is invertible, rearranging gives the **normal equation**:

$$\hat{\boldsymbol{\theta}} \;=\; (\mathbf{X}^\intercal\mathbf{X})^{-1}\,\mathbf{X}^\intercal y.$$

It is a **closed-form** solution: a finite number of constants, variables and standard operations, with no infinite sums, no limits, no integrals, and no iteration.

Convexity is not a technicality to be waved through. It is also why gradient descent, in Lecture 4, can only stop at the answer this closed form gives: its steps vanish only where the gradient does, and convexity makes every such point the minimum. With a suitable learning rate, it gets there.

2.1 What the stationarity condition says geometrically

Read $\mathbf{X}^\intercal(\mathbf{X}\hat{\boldsymbol{\theta}} - y) = \mathbf{0}$ again: every column of $\mathbf{X}$ is orthogonal to the residual $\mathbf{r} = \mathbf{X}\hat{\boldsymbol{\theta}} - y$.

As $\boldsymbol{\theta}$ varies, $\mathbf{X}\boldsymbol{\theta}$ ranges over the column space of $\mathbf{X}$. Minimising $\|\mathbf{X}\boldsymbol{\theta} - y\|_2$ therefore finds the point of that subspace closest to y — and the closest point of a subspace to a vector is its orthogonal projection.

The sentence to remember

Fitting a linear model is projecting the label vector onto the span of the features. The residual is what the features cannot reach.

The notebook checks this rather than believing it. Eight numeric features, imputed and scaled, with a column of ones added for the intercept: if the projection argument is right, $\mathbf{X}^\intercal\mathbf{r}$ is numerically zero. "Small" is meaningless until you divide by something, so the check divides the largest entry by the scale of y and asks for a ratio below 10^{-6}; it comes out at the level of machine precision.

2.2 When it breaks

The equation requires $(\mathbf{X}^\intercal\mathbf{X})^{-1}$ to exist. $\mathbf{X}^\intercal\mathbf{X}$ is $(n{+}1) \times (n{+}1)$ and is invertible if and only if it has full rank — equivalently, if and only if $\mathbf{X}$ has linearly independent columns. There are exactly two ways to lose that.

Failure condition — more features than instances

If $m < n + 1$ then $\text{rank}(\mathbf{X}) \leq m < n + 1$, so $\mathbf{X}^\intercal\mathbf{X}$ is singular. Note what the problem is: not that no parameter vector fits the data, but that infinitely many fit it equally well. Common in genomics, in text, and in any wide-feature setting.

Failure condition — collinear features

If one column is a linear combination of the others, the columns are dependent and $\mathbf{X}^\intercal\mathbf{X}$ is singular again.
This is Lecture 2's warning, now explained. "When creating new combined features, avoid simple weighted sums of existing features" — because a weighted sum of existing columns *is* a linear combination, and you would be manufacturing the singularity yourself.

Exact collinearity is rare; near-collinearity is common, and is also a problem. The matrix is invertible but ill-conditioned: tiny changes in the training set produce large changes in $\hat{\boldsymbol{\theta}}$, and the model is numerically unstable. Lecture 4 meets a rank-deficient design on the Titanic and repairs it by removing the redundant columns; the condition number is the diagnostic to watch.

2.3 What Scikit-Learn actually does

It does not invert anything. It computes $\hat{\boldsymbol{\theta}} = \mathbf{X}^+ y$, where $\mathbf{X}^+$ is the Moore–Penrose pseudoinverse, obtained from the singular value decomposition:

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\intercal \qquad \Longrightarrow \qquad \mathbf{X}^+ = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^\intercal.$$

To build $\boldsymbol{\Sigma}^+$: take $\boldsymbol{\Sigma}$, set to zero every value below a small threshold, replace the rest by their reciprocals, and transpose.

The pseudoinverse is always defined, even when the inverse is not. But notice what the threshold is doing. When the inverse does not exist there are infinitely many $\boldsymbol{\theta}$ that fit equally well; zeroing the tiny singular values selects one of them — the minimum-norm least-squares solution, the shortest vector among all the best fits.

A choice, not a discovery

That selection is a convention, not a fact about the data. It is why two rank-deficient fits can agree on every prediction and disagree on every coefficient, and why interpreting coefficients from a collinear design is unsafe even when the predictions are fine.

Examinable: derive the normal equation; state and justify the invertibility condition.

3 Measuring generalisation

3.1 A number that means nothing

Fit an unconstrained decision tree on the 16,512 training districts and score it on those same districts: RMSE 0.0. No error at all. There are two explanations and only one is flattering.

The model is perfect. Housing prices would have to be an exact deterministic function of

nine census attributes, with two identical districts always carrying identical median prices, no measurement noise anywhere in a census, and no influence from anything outside our nine columns. Reject.

The model memorised. An unconstrained tree keeps splitting until every leaf is pure, and with enough depth it isolates every training district into its own leaf. Predicting the training set is then a lookup, and a lookup has zero error.

Failure condition — a score computed on the rows it was fitted to

Any score computed on the rows a model was fitted to measures how well it reproduces what it has already seen. For a model flexible enough to memorise, that is not evidence about anything else — and nothing in the number tells you whether your model is flexible enough.

3.2 "Then use the test set"

No — not yet. We are still choosing between models and tuning them. Every time a choice is made by looking at the test score, that score stops being an estimate of performance on unseen data and becomes a target we are fitting to. The test set is used *once*, at the end, to estimate the generalisation error of a system already committed to.

3.3 Leakage: the same error, one step earlier

A score can be corrupted before any model is fitted.

```
scaler = StandardScaler()
housing_scaled = scaler.fit_transform(housing_full)   # all 20,640 rows
train_set, test_set = train_test_split(housing_scaled)  # then split
```

Definition 3.1 (Data leakage)**.** *A quantity computed from rows outside the training set enters the training path.*

Here the mean and standard deviation were computed from a set that includes the test rows, so the model is evaluated on rows whose own values helped define the transformation applied to them. It raises no exception, and the reported score improves. How much it improves is not knowable from the code — sometimes nothing, sometimes everything — which is why the rule is procedural rather than a judgement made case by case.

The rule, stated once

Fit every transformer on the *training set only*. Never call `fit()` or `fit_transform()` on the validation set, the test set, or new data. Once fitted, you may `transform()` anything.
This applies to imputers, scalers, encoders, dimensionality reduction — everything that learns anything from data. We will not rely on remembering it; the pipeline below makes violating it structurally impossible.

4 Cross-validation and pipelines

4.1 From holdout to k-fold

Hold out a **validation set** (also called the **development set**) from the training set: train candidates on the reduced set, select on the validation set, retrain the winner on the full training set, evaluate once on the test set. The difficulty is sizing it. Too small and the evaluations are imprecise, so you may select a poor model by chance; too large and the remaining training set is much smaller than the one the final model will use — like selecting a sprinter to run a marathon.

The resolution is many small validation sets, used in rotation. Split the training set into k non-overlapping **folds**, then train and evaluate k times, holding out a different fold each time:

$$\text{score} \;=\; \frac{1}{k}\sum_{j=1}^{k} \text{RMSE}(\text{model trained without fold } j,\ \text{fold } j)\,.$$

Every row of that procedure is a complete, independent fit. The cost is k fits. The return is not only a mean but a *spread*, which a single holdout cannot give at any price.

Two details in the call that are not decoration

```
cv = KFold(n_splits=10, shuffle=True, random_state=42)
rmses = -cross_val_score(model, X_train, y_train,
            scoring="neg_root_mean_squared_error", cv=cv)
```

The minus sign: Scikit-Learn's cross-validation expects a utility (greater is better), so the scorer is the *negative* RMSE and the sign is flipped back.
The explicit `KFold`: passing `cv=10` takes ten contiguous blocks in row order. Two people whose frames are sorted differently then get different folds and cannot see why their numbers disagree.

4.2 Ten folds, not one number

Linear regression, the unconstrained tree and a random forest, each cross-validated on the same ten folds, give ten RMSEs apiece. The tree that scored 0 on its own training rows is no better, measured this way, than the linear model. And within each model the folds spread by roughly \$2,000–\$3,000 — mostly a matter of *which districts landed in the fold*, not of the model.

What the spread licenses

Any difference between two models smaller than the fold-to-fold spread is not a difference. Report the folds, or at least their spread, beside the mean: a spread cannot be recovered from an average.

4.3 Pipelines make leakage structurally impossible

```
full_pipeline = Pipeline([
    ("prep",  preprocessing),   # imputer + scaler on numbers, one-hot on ocean_proximity
    ("model", RandomForestRegressor(random_state=42)),
])
```

When `cross_val_score` holds out fold j, the *entire* pipeline — imputer, encoder, scaler, model — is fitted on the other folds only. The imputer's median and the scaler's mean never see the held-out fold. Leakage cannot occur by accident, which is a stronger guarantee than any amount of care.

5 The test set, once

```
final_model = Pipeline([
    ("prep",  preprocessing),
    ("model", RandomForestRegressor(n_estimators=200, max_features=8,
                                    random_state=42)),
]).fit(X_train, y_train)                 # the training set, and only that
final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)   # $49,037
```

The forest's two settings were chosen on validation folds of the training set before this cell ran. We `predict` on the test set; we never `fit` anything on it, and we look at it once.

Failure condition — the temptation, and why it is fatal

Whatever the test number turns out to be, you must *not* tweak hyperparameters to improve it. Those improvements would not generalise; you would be overfitting the test set, which is the only thing standing between you and a confident fiction.

6 A classification task, and a first detector

6.1 The brief

A national postal operator sorts handwritten postcodes automatically: a camera photographs each digit, a classifier reads it, the envelope goes down a chute. Their audit team has found that one glyph is misread far more often than the others. They want every scanned digit that is a **5** pulled off the line and sent to a human verification desk.

Note what that is not. It is not "read the digit". It is *detect one digit* — yes or no, one class against the other nine.

6.2 The data

MNIST: 70,000 images of handwritten digits, written by US high-school students and Census Bureau employees, each 28×28 pixels flattened to 784 features, each feature one pixel's intensity from 0 (white) to 255 (black). It is studied so often that it is effectively the field's control experiment: if a method cannot do something here, the something is probably not the method's fault. The split is standard — the first 60,000 rows train, the rest test.

Relabelling for the task is two lines, `y_train_5 = (y_train == 5)`, and it creates the imbalance: 5,421 positives out of 60,000. The ten digits appear in roughly equal numbers; the *task* does not.

This is not a fact about MNIST

Every binary detector carved out of a multiclass problem inherits this. Detect one product fault out of forty; one disease out of a hundred; one fraudulent transaction in ten thousand. The arithmetic is identical and only the ratio changes. Nothing about MNIST is imbalanced. Our *question* is.

Failure condition — labels that arrive as strings

OpenML delivers the labels as one-character strings: `'5'`, not `5`. `y == 5` is then `False` for every image, with no error and no warning, because a string is never equal to an integer. The repair is a cast, `y = y.astype(np.uint8)`, and an assertion on the count, `assert y_train_5.sum() == 5421`: if the cast was skipped, the assertion fails at $0 = 5421$ instead of silently training on nothing.

6.3 The obvious metric, and the headline number

The obvious metric is **accuracy**, the fraction of instances the classifier gets right. It is between 0 and 1, bigger is better, it needs no threshold and no scaling, and every stakeholder understands it without being taught.

An `SGDClassifier` behind a `StandardScaler`, in one pipeline and three-fold cross-validated, scores **96.90%** — folds of 96.83%, 96.84% and 97.04%, agreeing to about two tenths of a point.

6.4 Before you commit: what does nothing score?

The cheapest possible detector looks at nothing and answers "not a 5" every time. It is right 54,579 times out of 60,000: **90.96%** accuracy, with no model, no fit and no features. Every number needs a baseline: a metric with nothing to compare it to is decoration.

7 The mathematics: what nothing scores, and what precision does

7.1 The classifier that never fires

Write m for the number of instances, P positives and N negatives with $P + N = m$, and $p = P/m$ for the **base rate**. Write TP, FP, FN and TN for the counts of true positives, false positives, false negatives and true negatives.

Claim

Let $h_0(x) = 0$ for every x. Then TP = FP = 0, so every negative is a true negative and

$$\text{accuracy}(h_0) \;=\; \frac{0 + N}{m} \;=\; \frac{N}{m} \;=\; 1 - p.$$

An identity, not an estimate: no model, no data, no variance. On our task $p = 0.09035$, so $\text{accuracy}(h_0) = 0.90965$. That is the 90.96% we measured, and it could have been written down without running anything.

Failure condition — accuracy rewards doing nothing on a rare event

As $p \to 0$, accuracy$(h_0) \to 1$: at a base rate of 1% doing nothing scores 99%. For any target accuracy below 1 there is a detection problem rare enough that the empty model beats it. So a high accuracy is not a statement about a classifier. It is a statement about a classifier *and* a base rate, and quoting it without the second is quoting half a measurement.

7.2 What the detector computes

The `SGDClassifier` is linear. It scores a digit with $s(x) = \boldsymbol{\theta}^\top \mathbf{x}$ on the scaled pixels — the same linear function as regression, with the bias folded in as $x_0 \equiv 1$ — and predicts a 5 when $s(x) \geq 0$. Regression reports the number; classification cuts it at a threshold. The first training image is a 5 with a large positive score: at threshold 0 it is flagged, at threshold 3000 the same 5 is missed.

`predict()` hides the number and hard-codes the threshold at zero. There is no `set_threshold()` method, and there should not be: you compute the scores with `decision_function` and compare them yourself. Everything below uses one out-of-fold score per training digit, from `cross_val_predict(. . . , method="decision_function")`.

7.3 Recall is monotone in the threshold; precision is not

Now let the threshold move. Write TP(t), FP(t), FN(t) and TN(t) for the counts at threshold t. Every scoring classifier is really a family of classifiers, one per threshold: $\hat{y}_t(x) = \mathbf{1}[s(x) \geq t]$. Raising t makes it more reluctant to fire. Nothing about the model changes — only where we cut.

The flagged set $\{x : s(x) \geq t\}$ shrinks as t rises, and it shrinks by losing instances, never by gaining any. So for $t' \geq t$, both TP$(t') \leq$ TP(t) and FP$(t') \leq$ FP(t).

$$\text{recall}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FN}(t)} = \frac{\text{TP}(t)}{P}.$$

The denominator does not depend on t at all: TP + FN counts every positive in the data, whatever we predict about it. So recall is a non-increasing quantity divided by a constant, and recall is non-increasing.

Precision has no such guarantee:

$$\text{precision}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FP}(t)},$$

where numerator and denominator both move, in the same direction. A ratio of two decreasing quantities can do anything at all. The denominator is the size of the set *we chose to flag*: not a property of the data, a property of our decision.

One step of the threshold

Raise t just enough to drop the lowest-scoring flagged instance. Write $T = \text{TP}$, $n = \text{TP} + \text{FP}$ before the step, and let $y \in \{0, 1\}$ be the true label of the instance dropped. Precision goes from T/n to $(T - y)/(n - 1)$, and cross-multiplying, the step increases precision exactly when

$$y \; < \; \frac{T}{n} \; = \; \text{precision before.}$$

Since y is 0 or 1: dropping a negative raises precision, always; dropping a positive lowers it, unless precision was already 1. On our own ranking — every training instance sorted by its out-of-fold score, the threshold walked down one instance at a time — that classifies all 59,999 steps: precision rises at 54,579 of them, falls at 5,417, and stays at 1 at 3. At the very top of the ranking the sixth-ranked digit is a 5, and raising the threshold past it takes precision from $5/6$ down to $4/5$.

8 What 96.90% was hiding

8.1 The confusion matrix

	Predicted not-5	**Predicted 5**
Actually not-5	53,957	622
Actually 5	1,237	4,184

Rows are the actual class, columns the predicted class; the diagonal is what went right. Both mistakes are visible *separately*, and they are not the same size or the same kind. Accuracy added them together and divided by 60,000. 1,237 fives went past unflagged: 22.8% of every 5 in the training set. Nothing new was fitted; we asked the same out-of-fold predictions a different question.

8.2 Precision, recall and F_1

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Recall is also called **sensitivity** or the **true positive rate**; three names, one quantity, and the third returns on the ROC curve.

Metric	**Value**	**Reads as**
Precision	87.06%	of the digits we flag, 87% really are 5s
Recall	77.18%	of the 5s that exist, we find 77%
Accuracy	96.90%	—

When one number is unavoidable, F_1 is the harmonic mean:

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Ours is 81.82%. The harmonic mean is dominated by the smaller of the two, so a high F_1 needs both to be high. A classifier that flags three digits a year and is right every time has an arithmetic mean above one half and an F_1 near zero.

9 The threshold, and the curves

9.1 You cannot have both

Raising recall lowers precision, and lowering recall raises it — usually. "Usually" is doing real work in that sentence, and the step count above says exactly how much. The trend is reliable; the individual step is not. The **precision/recall trade-off** is not a property of our model, of SGD or of MNIST. It is a property of ranking instances and cutting the ranking somewhere.

9.2 Choosing a threshold deliberately

Plotted against recall, our precision starts to fall off a cliff at about 80% recall; choose the operating point just before the cliff, not on it. Choosing 90% precision on cross-validated training scores gives threshold 20.09, at precision 0.9001 and recall 0.7333 — ninety per cent precision costs 3.86 points of recall, from 77.18% down to 73.33%.

The code asks for the lowest threshold at which precision reaches 0.90 *and* at least 500 digits are flagged. The second condition is there because precision is not monotone: "the first index reaching 90%" could be a single lucky step held up by a handful of flagged digits. The idiom is `np.argmax` on a boolean array, which returns the *first* `True`.

A threshold is a hyperparameter, so it is chosen on cross-validated training scores, never on the test set — choosing it there would make the test score optimistic, like any hyperparameter chosen on the test set.

9.3 ROC, and when not to use it

Same sweep, different axes:

$$\text{TPR} = \frac{\text{TP}}{P} = \text{recall}, \qquad \text{FPR} = \frac{\text{FP}}{N} = 1 - \text{specificity}.$$

Ours scores ROC AUC 0.9724. Note that *both* denominators are fixed by the data. That is precisely why the ROC curve behaves so much better than the PR curve — and why that is a warning rather than a recommendation.

The rule: prefer the precision/recall curve whenever the positive class is rare, or whenever you care more about false positives than false negatives. The reason is arithmetic — FPR divides false positives by N, which is huge when positives are rare, so a large number of false alarms produces a small, comfortable-looking FPR, while precision divides the same false positives by the flagged count, which is small.

Take our scores unchanged and thin the positive class, so the model, the ranking and the scores are identical and only the class balance moves. Average precision, scikit-learn's

`average_precision_score`, is the mean of the precision at the rank of each positive, so a random ranking scores about the base rate:

Base rate	ROC AUC	Average precision
9.04% — as measured	0.9724	0.8784
2.00%	0.9699	0.7203
1.00%	0.9709	0.5800

Failure condition — ROC on a rare positive class

Three indistinguishable ROC curves and three obviously different PR curves, from one set of scores. The false-positive rate divides by the abundant class, so the ROC curve cannot see what a rare positive costs — and it looks reassuring while it fails to.

9.4 A better classifier

A forest has no `decision_function`. It has `predict_proba`, whose positive-class column plays exactly the role of the score, and every curve above works unchanged.

Metric	SGD	Random forest
Accuracy	96.90%	98.77%
Precision	87.06%	98.93%
Recall	77.18%	87.31%
F_1	81.82%	92.76%
ROC AUC	0.9724	0.9984
Average precision	0.8784	0.9883
Recall available at 90% precision	73.33%	97.51%

Accuracy improves by 1.87 points, which reads as polish. Recall at 90% precision improves by 24.18 points, and it is the row the brief is about. Which description you report decides whether anyone approves the change.

10 The notebook

The notebook runs on a free CPU and stands alone: it downloads both datasets itself. Part 1 solves the normal equation and checks the residual orthogonal to every column, scores four models on their own training rows, cross-validates three of them on ten folds inside one pipeline, and uses the test set once. Part 2 builds the never-fires baseline and the detector, counts the non-monotonicity of precision at every one of the 59,999 thresholds, and draws the curves, the threshold choice and the forest.

What to change

Run it once, then detect a different digit: change one character — the 5 in `y_train == 5` — and re-run from that cell. The baseline moves, the accuracy moves with it, and the pair tells you something neither number does alone.

11 Design exercise: a payments app's real-time fraud engine

The lecture closes with a design exercise in the manner of an ML system-design interview, on a rare-event detection task of the kind the second half of the lecture was about.

The task. A payments app — a card in the customer's phone — must decide, in the fraction of a second between card and terminal, whether to approve or decline the transaction, or ask the customer "was this you?". The company reimburses the fraud it approves, as payment-services rules require, and loses the customer whose dinner it declines; before the engine goes live it wants to know how many frauds it catches and how many honest customers it turns away. Given: the authorisation log — amount, merchant, country, channel, time, the card's recent history — and the disputes customers raise in the app afterwards, which mark the frauds. Millions of transactions a day; fraud of the order of one in a thousand. The output feeds the authorisation service, the fraud team that works the flagged queue, and the customer who gets the push. The model at the centre is a classifier that produces a score, cut at a threshold chosen on purpose, and read through precision and recall rather than accuracy.

1. **Frame it.** Binary detection, fraud against everything else, with a positive class rare by construction. Before any model, the classifier that never fires: it flags nothing, and its accuracy is one minus the base rate — at one fraud in a thousand, about ninety-nine point nine per cent. The metric is precision and recall at the operating point the product chooses, with the PR curve behind it. *Pitfall:* a first model reports accuracy in the high nineties and is approved, when doing nothing scores higher and a model that catches every fraud while declining one honest customer in a hundred scores lower than doing nothing. Quote the never-fires accuracy first, and judge every model by the frauds it catches at a precision the fraud team and the customers can afford.

2. **The label arrives weeks later.** No transaction is born labelled: a fraud becomes known when the customer disputes it in the app and the fraud team confirms it, days or weeks after the authorisation, so the newest rows in the log are the least trustworthy negatives. Write the definition down — a confirmed dispute within the window; a refund for a faulty product is not one; a fraud the customer never notices is one the label will never show. *Pitfall:* the training set is cut from the newest rows of the log, its labels mostly unmatured, so frauds not yet disputed sit among the negatives and the quoted recall counts only the frauds known by then. Label only transactions older than the dispute window, record the date each label arrived, and say which frauds the recall counts.

3. **Evaluate in the order the engine will face.** The score must estimate performance on transactions that come after the model is fitted: train on earlier months, validate on the next, test on the one after, with a gap as long as the dispute window. Folds are consecutive periods, each scored by a model fitted on the periods before it, and each period's precision and recall are reported. The test period is read once. *Pitfall:* a shuffled k-fold puts a stolen card's ten transactions of one afternoon on both sides, so the model is scored on frauds whose siblings it trained on; recall climbs, and on the first live day falls back. Split by time, keep each card on one side, and treat the drop from the shuffled score to the time-ordered one as a measurement.

4. **Features as of the moment of authorisation.** The transaction's amount, merchant category, country, channel and hour; the card's recent spend and count, time and distance since the

previous transaction, whether the merchant is new to the card; scaled inside the pipeline, because the linear classifier is fitted by descent. A feature the authorisation service cannot compute in the time it has is not a feature. *Pitfall, leakage:* "transactions on this card today" computed over the whole day lets a row at noon see the evening; "the merchant's fraud rate" computed over the whole log contains each row's own label; and the card's *frozen* flag, set by the fraud team after the fraud, is the label in disguise. Compute every history feature from the past only, with the code that will run at authorisation, and fit any rate inside the pipeline on the training periods alone.

5. **The model: a score, not a label.** A linear classifier on the scaled features scores a transaction with $\boldsymbol{\theta}^{\top}\mathbf{x}$; `predict()` cuts at zero, so ask for `decision_function` instead, out of fold. A forest is the challenger on the same periods, its `predict_proba` column the score. Compare the two on recall at the affordable precision, not on accuracy. *Pitfall:* the forest is scored on the transactions it was fitted on and reports no error at all, because an unconstrained tree isolates every row in its own leaf and a lookup has zero error. Every score the team sees comes from a fit that never saw that period, with the whole pipeline refitted inside each fold.

6. **The threshold is a product decision.** A false positive costs a push, an embarrassed customer, a card at the back of the wallet and a case for the queue; a false negative costs the amount, reimbursed by law, and a customer who learns of it from a statement. The fraud team can work a bounded number of cases a day, so the flagged count is a constraint, and the threshold is chosen on the validation periods' out-of-fold scores to meet it. *Pitfall:* `predict()` ships with the cut at zero, a decision nobody took, and "the first threshold that reaches the required precision" can be one lucky step held up by a handful of flagged rows, because precision is not monotone. State the constraint, ask for the first threshold that reaches it with enough flagged transactions behind it, and defend the choice as the product decision it is.

7. **Evaluate on the curve that can see a rare positive.** The PR curve and its average precision, per period; the recall available at the required precision is the number the product signs off, reported per fold so that a quiet month and a fraud wave are both visible and a difference smaller than the spread is not a difference. *Pitfall:* the ROC AUC is superb, because its false-positive rate divides the false alarms by millions of honest transactions, so a thousand declined customers a day is a rounding error on that axis; the same scores at three base rates draw one ROC curve and three different PR curves. Prefer the PR curve whenever the positive class is rare or false positives are the costly error.

8. **In production, the engine chooses what gets labelled.** A declined transaction has no outcome, so a decline yields no label for the next model; the fraud team works the flagged queue, so confirmed frauds concentrate where the engine looked; and fraud adapts, so the base rate drifts, with the fraud team on call for the decline rate. *Pitfall:* the next model is trained on a log where the previous engine decided which rows carry a label, and its recall is measured against the frauds caught rather than the frauds that happened. Keep a small random sample of transactions approved and audited regardless of score, measure recall on it, stage each new model on a share of traffic against the old one, retrain on a schedule with matured labels, and re-choose the threshold whenever the base rate moves.

9. **Ship it.** Authorisation log → matured labels from confirmed disputes → split by time, each card on one side, a gap for the window → as-of features, scaled inside the pipeline → out-of-fold scores from a linear classifier and a forest → threshold chosen against the fraud

team's capacity and both costs → precision and recall per period, PR not ROC → a random audited sample, a staged rollout, retraining, the threshold re-chosen → the test period, once.

12 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 4's deck and in the exercise book.

1. The normal equation is $\hat{\theta} = (X^\top X)^{-1} X^\top y$. Your training matrix X has a column equal to the sum of two others. State what happens to $X^\top X$, and what the pseudoinverse X^+ returns instead. [5]

2. A pipeline scales the features and then fits a model, and is passed whole to `cross_val_score`. Explain what would go wrong if the scaling were done once, before the call. [5]

3. On the MNIST 5-detector, a classifier that never fires scores 90.96% accuracy on the training set. State what that number is a measurement of. [3]

4. Prove or disprove: as the decision threshold is lowered, precision increases monotonically. [5]

5. Two models have the same ROC AUC. One is far better on a precision–recall curve. What does that tell you about the problem? [4]

Summary

Fitting a linear model is orthogonal projection: the minimiser satisfies $\mathbf{X}^\intercal(\mathbf{X}\hat{\boldsymbol{\theta}} - y) = \mathbf{0}$, so the residual is orthogonal to every feature, and $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$ whenever the inverse exists. It fails to exist when $m < n + 1$ or when the features are collinear — which is why Lecture 2 warned against building features as weighted sums of existing ones. The SVD pseudoinverse is always defined, and silently chooses one of infinitely many fits when the problem is degenerate.

A score computed on the training rows cannot detect overfitting: an unconstrained tree scored 0. The test set is spent the first time it influences a choice. Leakage is a condition, not an accident, and a pipeline inside cross-validation makes it structurally impossible. k-fold gives an honest estimate *and* its spread, and a difference smaller than the spread is not a difference. The test set was touched once, at \$49,037.

On a rare-event classification task, the classifier that never fires scores exactly $1 - p$: 90.96% here, against our detector's 96.90%. Recall is non-increasing in the threshold because its denominator is P; precision's denominator is the flagged count, so one step raises precision exactly when the dropped instance is a negative. The same predictions gave precision 87.06% and recall 77.18%, with 1,237 fives past the desk. ROC AUC barely moved as the base rate fell from 9% to 1% while average precision fell sharply, which is why the PR curve is the one to read on a rare event.

The closing design exercise built the real-time fraud engine behind a payments app in nine steps — the never-fires anchor, labels that mature from disputes, a split by time, as-of features, out-of-fold

scores, a threshold priced in both errors, the PR curve on a rare positive, and a random audited sample against the engine's own feedback — each step named for the pitfall it avoids.